



Дії GitHub

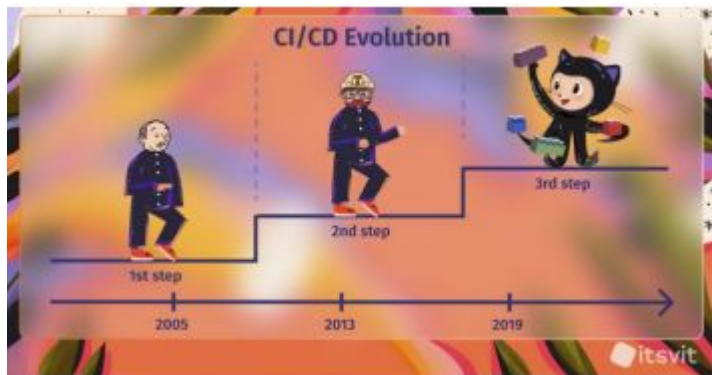
Вступ до CI/CD

Безперервна інтеграція (CI) та безперервне розгортання (CD) є основами сучасної розробки програмного забезпечення, які спрощують процес від фіксації коду до його розгортання. Неперервна інтеграція (CI) передбачає автоматизацію інтеграції змін коду від кількох авторів в один програмний проект. Цей процес включає автоматизоване тестування для швидкого виявлення помилок. CD автоматизує розгортання програмного забезпечення у виробничому середовищі, забезпечуючи швидку та надійну доставку функцій користувачам.

Переваги включають:

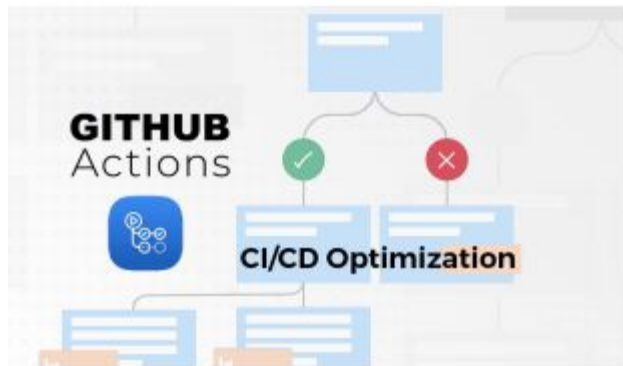
- Вища швидкість випуску: Автоматизуйте тестування та розгортання для швидшої ітерації.
- Вища якість: раннє виявлення та усунення помилок покращує якість програмного забезпечення.
- Ефективна співпраця: спрощує внесок розробників та зворотний зв'язок.
- Зниження ризику: менші, поступові оновлення зменшують ризик розгортання.

Еволюція CI/CD



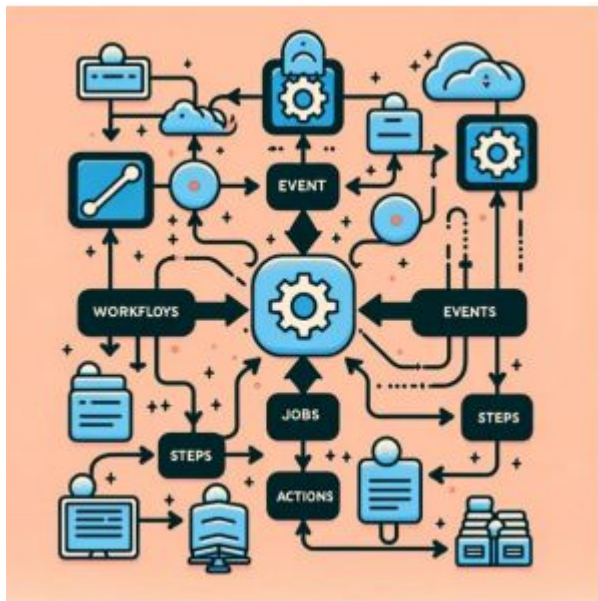
- До ери неперервної інтеграції: інтеграція була ручною, нечастою та схильною до помилок, що призводило до «інтеграційного пекла».
- Впровадження безперервної інтеграції: Концепція безперервної інтеграції виникла на початку 2000-х років, сприяючи регулярній інтеграції коду та автоматизованому тестуванню для раннього виявлення проблем.
- Поява CD: Безперервне розгортання (Continuous Deployment), що автоматизувало процес розгортання для забезпечення швидкої та надійної доставки змін коду у виробничі середовища.
- Інновації та інструменти: Впровадження складних інструментів CI/CD (таких як Jenkins, Travis CI, GitHub Actions) сприяло цим практикам, роблячи їх доступнішими та налаштованішими.
- Інтеграція з Agile та DevOps: практики CI/CD стали невід'ємною частиною методологій Agile та DevOps, підкреслюючи швидкі, ефективні та спільні цикли розробки.

Вступ до дій GitHub



GitHub Actions — це потужна платформа автоматизації, яка інтегрується безпосередньо в GitHub, дозволяючи розробникам автоматизувати робочі процеси з програмним забезпеченням безпосередньо зі своїх репозиторіїв. Запущений у 2018 році, він ознаменував значну еволюцію в екосистемі CI/CD, зробивши автоматизацію доступною в тому ж середовищі, де зберігається та перевіряється код. За допомогою GitHub Actions розробники можуть створювати власні робочі процеси для збирання, тестування та розгортання свого коду на основі різноманітних тригерів, таких як push-події або pull-запити. Ця інтеграція спрощує процес розробки, зменшує ймовірність помилок та значно пришвидшує доставку додатків. Його значення полягає в здатності демократизувати автоматизацію для розробників, надаючи гнучку, просту платформу, яка підтримує широкий спектр мов програмування та фреймворків, тим самим підвищуючи ефективність та надійність процесів розробки програмного забезпечення по всьому світу.

Основи дій GitHub



Дії GitHub дозволяють автоматизувати робочі процеси у ваших репозиторіях GitHub. Робочий процес, визначений у YAML-файлі, автоматизує такі процеси, як створення, тестування та розгортання коду. Ці робочі процеси запускаються подіями — певними діями у вашому репозиторії, такими як push-запити або pull-запити. У кожному робочому процесі ви визначаєте завдання, які є наборами кроків, що виконуються на виконавцях (віртуальних машинах). Кроки – це окремі завдання, які можуть виконувати команди або дії. Дії, найменші одиниці робочого процесу, – це фрагменти коду, які можна повторно використовувати та виконують певне завдання, таке як налаштування середовища розробки або розгортання на сервері. Цей фреймворк дозволяє розробникам автоматизувати процеси розробки програмного забезпечення, роблячи їх більш ефективними, послідовними та стійкими до помилок.

Компоненти дій GitHub



- **Робочий процес:** Центральний елемент GitHub Actions, він визначає процес автоматизації.
- **Тригер:** Визначає події, які ініціюють робочі процеси, такі як push-запити або pull-запити.
- **Завдання:** Являє собою послідовність завдань, які мають бути виконані, часто паралельно або послідовно.
- **Крок:** Окреме завдання в рамках роботи, таке як запуск скрипта або розгортання коду.
- **Дія:** Повторно використовувані одиниці роботи, визначені у файлах YAML, що дозволяють налаштування та інтеграцію.
- **Виконавець:** Середовище, де виконуються робочі процеси, що забезпечує необхідні ресурси та інструменти.
- **Артефакт:** Дані, створені робочим процесом, такі як результати збірки або тестування, які часто зберігаються для подальшого використання або довідки.

Детальний опис робочих процесів

Робочі процеси є основою акцій GitHub, виконуючи роль автоматизованих процедур, визначених у репозиторії для обробки завдань розробки програмного забезпечення. Робочий процес складається з одного або кількох завдань і запускається певними подіями, такими як надсилання до гілки, запит на зняття змін або за розкладом. Кожен робочий процес описано за допомогою синтаксису YAML у каталозі `.github/workflows` репозиторію GitHub. Така конфігурація дозволяє автоматизувати тестування, збірку та розгортання коду залежно від визначеної послідовності завдань. Робочі процеси можуть виконувати завдання одночасно або послідовно, а також їх можна налаштувати для виконання на різних типах раннерів, включаючи раннери, розміщені на GitHub, або самостійно розміщені раннери. Використовуючи робочі процеси, розробники можуть значно зменшити ручне втручання, забезпечуючи послідовне та ефективне виконання процесів розробки.

Події, що запускають робочі процеси

- **Зміни коду:** Події, що викликаються коммітами коду або злиттям у репозиторій.
- **Запити на зняття:** ініціювати робочі процеси, коли запити на зняття відкриваються, закриваються або оновлюються.
- **Проблеми:** Події, пов'язані зі створенням проблеми, коментарями або оновленнями, можуть запускати робочі процеси.
- **Відправлення репозиторію:** Користувацькі події, що запускаються ззовні для запуску робочих процесів.
- **Заплановані події:** Робочі цикли можна запланувати на виконання у певний час або з певними інтервалами.
- **Зовнішні події:** Інтеграція із зовнішніми службами або системами може запускати робочі процеси.
- **Ручні тригери:** Робочі процеси можуть бути запуснені вручну користувачами через інтерфейс GitHub.

Робота та паралелізм

Завдання слугують одиницями роботи в рамках робочих процесів, кожна з яких містить набір кроків для виконання. За замовчуванням завдання виконуються послідовно, одне за одним. Однак, робочі процеси можна оптимізувати для підвищення продуктивності, запускаючи кілька завдань одночасно, практика, відома як паралелізм.

```
name: first workflow

on: [push]

jobs:
  first-job:
    runs-on: ubuntu-latest
    steps:
      - name: Listing contents before checkout
        run: |
          pwd
          ls

      - name: checkout code
        uses: actions/checkout@v3

      - name: Listing contents after checkout
        run: |
          ls
```

Кроки: Робочі одиниці

Кожен крок представляє окреме завдання, яке може виконувати команди, скрипти або дії в межах завдання. Кроки виконуються послідовно в контексті завдання, що дозволяє виконувати серію завдань упорядковано. Така структура дозволяє розробникам створювати детальні процеси, такі як компіляція коду, виконання тестів або розгортання програм, у чіткому, покроковому робочому процесі. Кроки можуть використовувати дії, які є фрагментами коду повторного використання, для виконання певних завдань, що робить робочі процеси більш модульними та простішими в обслуговуванні. Ефективно використовуючи кроки, розробники можуть створювати точні та ефективні пайплайни CI/CD, адаптовані до потреб свого проекту.

```
name: environments workflow

on: [push]

jobs:
  windows-job:
    runs-on: windows-latest
    steps:
      - name: using powershell
        shell: pwsh
        run: Get-Location
  macos-job:
    runs-on: macos-latest
    steps:
      - name: running in mac
        shell: bash
        run: echo Hello world
  linux-job:
    runs-on: ubuntu-latest
    steps:
      - name: running in ubuntu
        shell: bash
        run: echo Hello world
```

Дії: Частина коду, які можна використовувати повторно

Дії в GitHub Дії – це фрагменти коду, які можна використовувати повторно та вбудовувати в кроки завдання, що спрощує створення та підтримку робочого процесу. Ці дії дозволяють розробникам інкапсулювати часто використововані команди, скрипти або утиліти в окремі блоки, якими можна ділитися в кількох робочих процесах або навіть зі спільнотою GitHub. Використовуючи дії, розробники можуть уникнути дублювання, забезпечуючи DRY (Don't Repeat Yourself) підхід до проектування робочого процесу. Така модульність не лише пришвидшує розробку конвеєрів CI/CD, але й сприяє впровадженню найкращих практик та співпраці, дозволяючи командам використовувати та робити внесок у зростаючу бібліотеку дій, доступну на GitHub Marketplace. Дії роблять робочі процеси ефективнішими, налаштовуванишими та легшими для оновлення, втілюючи принципи повторного використання коду та автоматизації.

Пояснення бігунів

Виконавці (runners) у діях GitHub — це сервери, які виконують завдання, визначені в робочих процесах (workflows). Кожен виконавець може бути віртуальною машиною, розміщеною на GitHub, або самостійно розміщеною машиною, якою ви керуєте. Бігуни оснащені певною операційною системою, такою як Linux, Windows або macOS, що дозволяє виконувати робочі процеси в середовищах, що відповідають їхнім цілям розгортання або вимогам тестування. Вони автоматично завантажують ваш код та виконують кроки й дії робочого процесу. Після завершення завдання, виконавець повідомляє про результати назад до GitHub. Така конфігурація забезпечує гнучкість та контроль над середовищем виконання, гарантуючи, що робочі процеси можуть виконуватися в умовах, що точно імітують виробниче середовище або відповідають конкретним потребам проекту, тим самим підвищуючи надійність процесу CI/CD.

Налаштування вашого першого робочого процесу

Налаштування вашого першого робочого процесу в GitHub Actions включає кілька простих кроків. Почніть зі створення каталогу `.github/workflows` у вашому репозиторії. У цьому каталозі створіть YAML-файл (наприклад, `main.yml`) для визначення вашого робочого процесу. Почніть файл з назви вашого робочого процесу та вкажіть, за яких подій він має спрацьовувати, наприклад, за запитами `push` або `pull` до головної гілки. Далі визначте завдання, які окреслюють завдання, що мають бути виконані, розбивши ці завдання на кроки. Кроки можуть включати налаштування середовища вашого проекту, проведення тестів та розгортання вашого коду.

```
name: CI

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run a one-line script
        run: echo Hello, world!
      - name: Run a multi-line script
        run: |
          echo Add other commands here
          echo This is a multi-line script
```

Синтаксис YAML для дій GitHub

YAML, або YAML Ain't Markup Language (YAML — це не мова розмітки), — це стандарт серіалізації даних, що читається людиною, що використовується в GitHub Actions для визначення робочих процесів. Це дозволяє розробникам описувати свої процеси автоматизації, включаючи тригери, завдання, кроки та дії, у структурованому форматі. YAML-файли для дій GitHub необхідно розмістити в каталозі `.github/workflows` репозиторію. Ключові елементи включають назву робочого процесу, пункт для визначення тригерних подій, завдання для опису завдань та кроки в цих завданнях, кожне з яких може використовувати дії. Відступи є критично важливими в YAML для позначення ієрархії та структури, причому пробіли зазвичай використовуються для відступів (табуляція заборонена). Читабельність та простота YAML роблять його ідеальним вибором для визначення складних робочих процесів у діях GitHub, що дозволяє розробникам легко автоматизувати свої конвеєри CI/CD.

Використання секретів та змінних середовища

Секрети та змінні середовища в GitHub Actions дозволяють безпечно керувати конфіденційними даними та налаштуваннями конфігурації у ваших робочих процесах. Секрети – це зашифровані змінні, що зберігаються на рівні репозиторію або організації, доступні в робочих процесах, але приховані від журналів і не доступні неавторизованим користувачам. Змінні середовища можна використовувати для встановлення параметрів конфігурації та визначати у файлі робочого процесу YAML або в налаштуваннях репозиторію. Щоб використовувати секрет у своєму робочому процесі, спочатку додайте його до налаштувань репозиторію в розділі «Секрети».

```
name: CI

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to server
        run: deploy_script.sh
        env:
          SERVER_API_KEY: ${ secrets.SERVER_API_KEY }
```

Впровадження матричних побудов

```
name: CI

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [10.x, 12.x, 14.x]
        os: [ubuntu-latest, windows-latest, macOS-latest]
    steps:
      - uses: actions/checkout@v2
      - name: Use Node.js ${matrix.node-version}
        uses: actions/setup-node@v1
        with:
          node-version: ${matrix.node-version}
      - run: npm install
      - run: npm test
    env:
      CI: true
```

Матричні збірки в GitHub Actions дозволяють запускати тести в кількох середовищах, визначаючи набір змінних, які генерують матрицю завдань. Це дозволяє паралельне виконання завдань у різних комбінаціях середовищ, таких як операційні системи, мови програмування або версії. Матрична стратегія скорочує час, необхідний для тестування, використовуючи паралельність, і гарантує, що ваш код працюватиме належним чином за різних умов.

Залежності кешування

Кешування залежностей у діях GitHub може значно скоротити час збірки, зберігаючи та повторно використовуючи частини середовища робочого процесу, такі як бібліотеки або пакети, між запусками. Під час виконання робочого процесу, замість завантаження залежностей з нуля, спочатку можна перевірити кеш. Якщо залежності не змінилися, робочий процес може використовувати кешовані версії, що пришвидшує процес.

```
name: CI

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Cache node modules
        uses: actions/cache@v2
        with:
          path: ~/.npm
          key: ${ runner.os }-node-${ hashFiles('**/package-lock.json') }
          restore-keys: |
            ${ runner.os }-node-
```

Планування робочих процесів

- Планування робочих процесів у GitHub Actions дозволяє автоматизувати виконання завдань у заздалегідь визначений час, використовуючи синтаксис cron для планування. Ця функція особливо корисна для рутинних завдань, таких як щонічні збірки, щоденні звіти або регулярне технічне обслуговування. Заплановані робочі процеси визначаються в полі `on` вашого файлу робочого процесу, що дозволяє точно контролювати, коли вони запускаються, незалежно від надсилання коду або запитів на зняття коду.
- У цьому прикладі налаштовується робочий процес на щоденне виконання о 2:00 UTC. Синтаксис cron `'0 2 * * *'` визначає хвилину (0), годину (2), день місяця (* для будь-якого), місяць (* для будь-якого) та день тижня (* для будь-якого), що дозволяє гнучко планувати для задоволення різних потреб автоматизації.

```
on:  
  schedule:  
    - cron: '0 2 * * *' # Runs at 2 AM UTC every day
```

Зовнішні дії: Торговий майданчик

GitHub Marketplace — це центральний хаб, де ви можете знаходити та використовувати дії, створені спільнотою GitHub та нею, покращуючи свої робочі процеси без необхідності винаходити велосипед. Ці дії охоплюють широкий спектр завдань автоматизації, від аналізу коду та сканування безпеки до розгортання. Використання зовнішніх дій спрощує створення робочих процесів і розширює можливості дій GitHub за допомогою рішень, розроблених спільнотою.

```
name: CI

on:
  schedule:
    - cron: '0 2 * * *' # Runs at 2 AM UTC every day

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Cache node modules
        uses: actions/cache@v2
        with:
          path: ~/.npm
          key: ${ runner.os }-node-${ hashFiles('*/package-lock.json') }
          restore-keys: |
            ${ runner.os }-node-
```

Команди робочого процесу для дій

```
echo "name=ACTION_STATE::success" >> $GITHUB_ENV
```

- Команди робочого процесу для GitHub Actions дозволяють діям взаємодіяти з машиною запуску GitHub Actions для встановлення змінних середовища, вихідних значень тощо безпосередньо зі скриптів або дій. Ці команди дотримуються певного синтаксису, що дозволяє динамічно та гнучко керувати виконанням робочого процесу.
- У цьому прикладі команда робочого процесу використовується для встановлення змінної середовища ACTION_STATE на успішне виконання дії в контексті виконання. Ця команда виконується шляхом повторення синтаксису команди у файлі \$GITHUB_ENV, який запускар дій GitHub обробляє для оновлення середовища для наступних кроків. Команди робочого процесу надають потужний спосіб маніпулювати середовищем робочого процесу та керувати потоком виконання на основі динамічних умов або результатів дій.

Корисні дані подій та вебхуки

- Корисні навантаження подій та вебхуки в GitHub Actions забезпечують механізм для робочих процесів реагування на широкий спектр подій GitHub, пропонуючи детальний контекст про подію, яка запустила робочий процес. Корисні навантаження містять дані у форматі JSON, пов'язані з подією, такі як SHA коміта, назва гілки або деталі проблеми, що дозволяє діям у робочому процесі приймати обґрунтовані рішення на основі контексту події.
- Цей робочий процес запускається надсиланням до головної гілки та включає крок, який друкує все корисне навантаження події. Завдяки доступу до `github.event`, робочий процес може динамічно налаштовувати свою поведінку, наприклад, розгортати в різних середовищах на основі вмісту корисного навантаження. Вебхуки, з іншого боку, дозволяють сповіщати зовнішні сервіси про ці події, що забезпечує інтеграції, які можуть реагувати на ці дії GitHub, ще більше розширюючи можливості автоматизації за межі власної екосистеми GitHub.

```
name: CI

on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v2
      - name: Print Push Event Payload
        run: echo "The event payload: ${ toJson(github.event) }"
```

Практики безпеки для дій GitHub

```
permissions:
  contents: read
  issues: write

jobs:
  example:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-node@v2
        with:
          node-version: '14'
      - run: npm install
      - run: npm test
```

- **Використовуйте секрети для конфіденційних даних:** зберігайте ключі API, облікові дані та інші конфіденційні дані як зашифровані секрети, ніколи не вбудовуйте їх жорстко у файли робочого процесу.
- **Обмеження дозволів:** Використовуйте ключове слово `permissions`, щоб обмежити дозволи токена GitHub, надаючи лише необхідний доступ для функціонування робочого процесу.
- **Перевіряйте дії третіх сторін:** використовуйте дії з перевірених джерел. Закріпіть дії до певного SHA, щоб уникнути шкідливих модифікацій.
- **Регулярно оновлюйте дії:** оновлюйте дії, щоб включати виправлення та покращення безпеки.
- **Моніторинг та аудит робочих процесів:** регулярно переглядайте свої робочі процеси та історію запуску на наявність незвичайної активності або попереджень безпеки.

Тестування дій GitHub

Тестування робочих процесів GitHub Actions та користувацьких дій гарантує, що вони працюють належним чином, виявляючи помилки на ранніх етапах процесу розробки. Ключові стратегії включають:

- Модульне тестування для користувацьких дій: Розробляйте модульні тести для ваших користувацьких дій, особливо якщо вони складні або мають критично важливу функціональність. Використовуйте фреймворки для тестування, що відповідають мові дії, такі як Jest для JavaScript.
- Робочий процес виконується на гілках функцій: Тестуйте робочі процеси, запускаючи їх на гілках функцій перед об'єднанням з вашою основною гілкою. Це дозволяє вносити необхідні корективи, не впливаючи на основний робочий процес.
- Інструмент Act для локального тестування: Використовуйте такі інструменти, як nektos/act, для локального запуску робочих процесів, імітуючи середовище GitHub Actions на вашому комп'ютері. Це пришвидшує процес тестування.
- Використання фіктивних секретів для тестування: налаштуйте свої робочі процеси за допомогою фіктивних секретів для тестування сценаріїв використання секретів, гарантуючи безпеку справжніх секретів.

Налагодження дій GitHub

Налагодження дій GitHub передбачає стратегічний підхід до виявлення та вирішення проблем у ваших робочих процесах. Один з ефективних методів — це ввімкнути покрокове ведення журналу, щоб отримати уявлення про процес виконання ваших дій. Це можна зробити, встановивши для секретного параметра `ACTIONS_STEP_DEBUG` у вашому репозиторії значення `true`, що дозволить детальне ведення журналу для кожного кроку ваших робочих процесів. Крім того, використання умовного виконання з операторами `if` може допомогти ізолювати проблеми, вибірково запускаючи частини робочого процесу на основі користувацьких умов.

Керування виконанням робочих циклів

Керування виконанням робочих процесів у GitHub Actions включає моніторинг їх виконання, скасування непотрібних або проблемних запусків та повторний запуск робочих процесів для усунення збоїв або внесення змін до тестування. GitHub надає інтерфейс користувача для цих завдань, доступний на вкладці «Дії» репозиторію. Моніторинг здійснюється легко через цю панель інструментів, де відображається стан і результати кожного запуску.

Щоб скасувати запущений робочий процес, можна скористатися інтерфейсом користувача, щоб вибрати певний запуск, а потім скористатися опцією «Скасувати робочий процес». Для повторного запуску робочих процесів GitHub пропонує можливість повторно запускати невдалі завдання або весь робочий процес з того ж інтерфейсу, що особливо корисно для вирішення тимчасових помилок або після внесення виправлень.

Сповіщення в діях GitHub

Сповіщення в GitHub Actions надають оновлення щодо стану виконання робочих процесів, допомагаючи командам бути в курсі своїх процесів автоматизації. Дії GitHub підтримують кілька типів сповіщень, включаючи сповіщення електронною поштою, веб-сайти та мобільні сповіщення, які можна налаштувати через налаштування репозиторію або файли робочого процесу. Для сповіщень, що стосуються робочого процесу, ви можете використовувати події on: [status] у файлі робочого процесу, щоб запускати сповіщення про певні результати (успіх, невдача, скасування). Крім того, інтеграція сторонніх служб сповіщень через дії на ринку дозволяє створювати налаштовані канали сповіщень, такі як Slack, Microsoft Teams або SMS.

```
name: Notify on Failure
on:
  push:
    branches:
      - main
jobs:
  build-and-notify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Build
        run: exit 1 # Simulate a build failure
      - name: Notify Failure
        if: failure()
        uses: some/notification-action@v1
        with:
          message: 'Build failed!'
          token: ${ secrets.NOTIFICATION_TOKEN }
```

Артефакти та журнали

У діях GitHub артефакти — це файли, що генеруються під час виконання робочого процесу, такі як бінарні файли, журнали або результати тестів, які можна зберігати та завантажувати після завершення завдання. Зберігання артефактів має вирішальне значення для налагодження та дозволяє обмін даними між завданнями та після завершення робочого процесу. Доступ до журналів надає уявлення про процес виконання, допомагаючи виправляти збої або неочікувану поведінку.

```
name: Build and Use Artifacts
on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Compile the project
        run: |
          mkdir build
          echo "Compiling project..." > build/output.txt
      - name: Upload artifacts
        uses: actions/upload-artifact@v2
        with:
          name: compiled-output
          path: ./build/

  use-artifacts:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Download artifacts
        uses: actions/download-artifact@v2
        with:
          name: compiled-output
      - name: Use the artifacts
        run: |
          ls -R
          cat compiled-output/output.txt
```

Питання та відповіді